INSTITUTO UNIVERSITÁRIO DE LISBOA

Mestrado em Engenharia Informática

Segurança em Redes e Sistemas de Informação

Segurança em Camadas para Comunicações IoT Baseadas em MQTT

Implementação, Avaliação num Raspberry Pi e Validação

Projeto — Parte 2: implementação e avaliação

Autores:

Rodrigo Gorjão, N.º 82972
João Afonso, N.º 97990
João Tendeiro, N.º 129087
Jossivalter Leal, N.º 146851

Docente: Daniel Maximino Caçador

Repositório: `https://github.com/joaoaeafonso/mqtt_secure`

Resumo

A primeira parte deste projeto propôs uma arquitetura de segurança em camadas para comunicações MQTT e um plano para a avaliar. Esta segunda parte apresenta o protótipo que a implementa, as medições feitas num Raspberry Pi 5 e a validação do protótipo perante os ataques previstos. Cada camada pode ser ligada e desligada de forma independente, o que permitiu medir o custo e o benefício de cada uma em doze configurações (K0 a K11).

O transporte (mTLS) e o envelope AES-GCM custam cada um 7 a 8 μs de processador por mensagem no *gateway*, e as restantes verificações, juntas, cerca de 2 μs; as assinaturas Ed25519 custam mais 151 μs e limitam o débito a 5 000 mensagens/s. O transporte e o envelope impedem, cada um, quase todos os ataques de um estranho, mas os de um dispositivo comprometido só são impedidos pela combinação de todas as camadas, que K8, K9 e K11 conseguem. A configuração recomendada é K8 para a telemetria (30 000 mensagens/s, p99 de 0,26 ms) e K9 para as ordens a atuadores, como a primeira parte previa. Das cinco hipóteses, três confirmam-se; refuta-se que o custo do TLS esteja no estabelecimento da ligação, e não se confirma que a limitação de tráfego poupe processador quando a criptografia é AES em *hardware*.

Palavras-chave: MQTT, Internet das Coisas, segurança em camadas, proteção fim-a-fim, avaliação de desempenho, Raspberry Pi.

Índice

Resumo **i**

1 Introdução **1**

2 Protótipo **1**
2.1 Visão geral . . . 1
2.2 O que foi implementado . . . 2
2.3 Decisões tomadas nesta parte . . . 2
2.4 Testes automatizados . . . 3

3 Método experimental **3**
3.1 Plataforma . . . 3
3.2 Configurações . . . 4
3.3 Métricas . . . 4
3.4 Ataques . . . 4
3.5 Procedimento . . . 5
3.6 Parâmetros fixados antes das medições . . . 6
3.7 Desvios ao plano . . . 6

4 Resultados: custo **7**
4.1 Custo por configuração (QP1) . . . 7
4.2 Custo marginal de cada camada . . . 8
4.3 Tamanho da mensagem e saturação (QP3) . . . 10
4.4 Aceleração criptográfica por *hardware* (QP4) . . . 11
4.5 Octetos e memória . . . 11
4.6 Plataforma de referência . . . 11

5 Resultados: proteção **11**
5.1 Ataques impedidos por configuração (QP2) . . . 12
5.2 Cada ataque contra a sua defesa, com controlo negativo . . . 13
5.3 Observações . . . 14

6 Critério de escolha e recomendação **15**

7 Hipóteses **16**

8 Validação do protótipo **17**
8.1 Conformidade com a especificação . . . 17
8.2 Validação do instrumento de medição . . . 18
8.3 Calibração da validade temporal . . . 18
8.4 Reinício do recetor . . . 19
8.5 Rotação e revogação . . . 19

9 Ameaças à validade e limitações **19**
9.1 Validade de construção 19
9.2 Validade interna 20
9.3 Validade externa 21
9.4 Limitações do protótipo 21

10 Conclusões **21**

Referências **23**

A Reprodução dos resultados **24**

1 Introdução

A primeira parte do projeto reviu a literatura sobre segurança em comunicações MQTT, propôs uma arquitetura em camadas (Parte 1, Secção 6) e definiu o plano para a avaliar (Parte 1, Secção 7). O objetivo desta segunda parte é o que esse plano enuncia: medir, num Raspberry Pi, quanto custa cada camada em desempenho e quanto protege, isoladamente e em combinação, para identificar a configuração com a maior proteção e o menor impacto no desempenho.

O documento responde às cinco perguntas experimentais da Parte 1:

QP1. Qual é o custo de cada camada num Raspberry Pi (processador, latência, débito, memória, octetos)? (Secção 4)

QP2. Que ataques impede cada camada, e quais só a combinação de camadas impede? (Secção 5)

QP3. Como varia o custo com o tamanho da mensagem e a carga, e quando satura o Raspberry Pi? (Secção 4)

QP4. Quanto pesa a aceleração criptográfica por *hardware*? (Secção 4)

QP5. Que configurações são ótimas de Pareto, e qual recomendar para cada caso de uso? (Secção 6)

Contributos desta parte.

- Um protótipo em C++23 da arquitetura proposta, com cada camada ativável de forma independente, e os seus testes automatizados (Secção 2).
- Um laboratório de carga que executa de forma automática as doze configurações, os ataques e os respetivos controlos negativos contra um *broker* real, e que guarda os dados brutos e o registo do ambiente de cada execução (Secção 3).
- As medições no Raspberry Pi 5 e num computador de secretária, a aplicação do critério de escolha e o veredicto das hipóteses H1 a H5 (Secções 4 a 7).
- A validação do protótipo e a calibração dos limites de validade temporal que a Parte 1 deixou em aberto (Secção 8).

O plano da Parte 1 foi seguido; o que foi concretizado de forma diferente, ou decidido só nesta parte, está declarado na Secção 3.7.

2 Protótipo

2.1 Visão geral

O protótipo tem três partes. A **biblioteca** (C++23) implementa as camadas como elementos de uma cadeia de processamento: cada mensagem que chega passa pela limitação de tráfego, pela camada de segurança (o envelope) e, opcionalmente, pela persistência; cada camada pode ser ligada e desligada em execução, sem reconstruir a cadeia. O ***gateway* de ensaio** liga essa

cadeia a um *broker* Mosquitto real através de um cliente Paho e reporta, por janela de medição, as mensagens entregues e rejeitadas (por motivo), a latência e o tempo de processador. O **laboratório de carga** (Python) lança o *broker*, o *gateway*, os sensores simulados e o atacante, aplica cada configuração e mede.

A Figura 1 mostra onde corre cada parte na campanha.

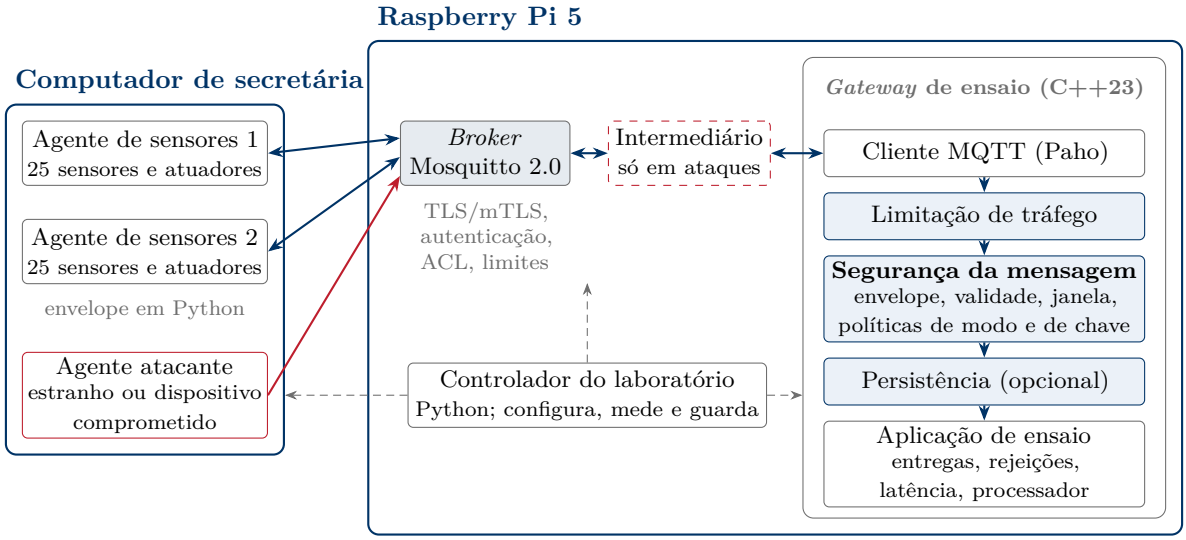


Figura 1. Arquitetura implementada, tal como corre na campanha. A cadeia de camadas da Parte 1 (Parte 1, Secção 6.1) está no *gateway* de ensaio. Setas contínuas: MQTT (telemetria dos sensores para o *gateway*, ordens no sentido inverso; a vermelho, o atacante). O intermediário só existe nos ensaios de ataques na rede. A tracejado: o canal de controlo do laboratório (TLS), que lança e configura cada parte e fica fora do caminho medido.

2.2 O que foi implementado

A Tabela 1 relaciona cada elemento da proposta com a sua concretização.

2.3 Decisões tomadas nesta parte

Reinício do recetor. A Parte 1 registou como limitação que a janela anti-repetição se perde quando o recetor reinicia. A solução adotada não persiste a janela a cada mensagem: o recetor guarda de forma durável apenas a época de chaves mais recente que aceitou e, depois de um reinício, só aceita épocas posteriores. A gravação faz-se uma vez por rotação, antes de aceitar a primeira mensagem da nova época, e uma falha de leitura ou de escrita rejeita a mensagem.

Dependências entre mecanismos. A validade temporal, a anti-repetição e a restrição chave–tópico verificam campos do envelope. Uma configuração que as ligue com o envelope desligado é recusada, no arranque e em execução, em vez de as ignorar em silêncio.

Fora do âmbito. Não foram implementados um modo HMAC próprio (o GMAC cobre a autenticação sem cifra), *nonces* por contador nem o modo de cifra seguida de assinatura, que a Parte 1 apresenta como possível extensão.

Tabela 1. Elementos da arquitetura proposta e a sua implementação.

Elemento (Parte 1)	Implementação	Onde
Envelope de 25 octetos; tópico e modo nos dados associados; modos de cifra, autenticação e assinatura (§6.4)	AES-256-GCM, GMAC e Ed25519 sobre OpenSSL; a mesma implementação em Python para os sensores, verificada pelos mesmos vetores de teste	`modules/security`, `msec_envelope.py`
ChaCha20-Poly1305 como alternativa (§6.4)	O algoritmo é um atributo da chave: o formato do envelope não muda e um recetor serve os dois	`chacha20_poly1305_cipher.cpp`
Ordem de verificação (§6.5)	Formato, modo aceite, validade, verificação criptográfica, chave–tópico, janela	`security_layer.cpp`
Validade temporal (§6.6)	Δ_a e Δ_s globais e por fluxo (filtro de tópico)	`FreshnessRule`
Janela anti-repetição, $w = 64$ (§6.6)	Por chave, em memória, com estado limitado	`replay_guard.cpp`
Restrição chave–tópico (§6.7)	Política com padrões por dono da chave, aplicada depois da autenticação	`key_policy.cpp`
Chaves por época, rotação e revogação (§6.8)	Rotação com período de graça; revogação de chaves, palavras-passe e certificados (CRL)	`pki.py`, `lab.py`
Limitação de tráfego antes da criptografia (§6.9)	*Token bucket* por tópico	`modules/limits`
Camada de transporte (§6.3)	TLS 1.3, autenticação mútua, palavras-passe, ACL por utilizador, identificador de cliente ligado ao utilizador, limites de pacote e de ligações	`broker.py` (Mosquitto)

2.4 Testes automatizados

O protótipo tem 93 testes em C++ (unitários, de integração e de aceitação), executados com AddressSanitizer e UndefinedBehaviorSanitizer, e 73 testes em Python (61 do laboratório, 12 da análise). Os testes de aceitação têm um teste por regra da ordem de verificação e foram verificados por mutação: cada um falha quando a regra correspondente é removida. Três alvos de *fuzzing* exercitam a análise do envelope, a validação de tópicos e todo o caminho de receção com octetos arbitrários. A integração contínua compila com `clang-tidy` (avisos tratados como erros) e corre os cenários de ataque contra um Mosquitto real.

3 Método experimental

3.1 Plataforma

A Tabela 2 descreve as duas máquinas. O Raspberry Pi 5 executa o *broker* e o *gateway*; o computador de secretária executa os sensores simulados (dois agentes) e o atacante (um agente). As duas máquinas estão ligadas por cabo, a 1 Gb/s, a um mesmo comutador Ethernet simples, sem router no caminho medido. No Raspberry Pi, o regulador de frequência estava em `performance`, o arrefecimento ativo oficial estava montado e os serviços alheios ao ensaio (incluindo o Docker) estavam parados durante a campanha.

O código medido é o do *commit* `79c4878`, o mesmo nas duas máquinas: o controlador recusa agentes de outra versão. A campanha correu de 12 para 13 de novembro de 2026 (4 h 55 min, sem intervenção), com o ambiente gráfico do Raspberry Pi desligado.

Tabela 2. Plataforma do nível 2.

	Raspberry Pi 5 (*broker* e *gateway*)	**Computador de secretária** (sensores e atacante)
Processador	Raspberry Pi 5 Model B rev. 1.1 (4 núcleos Cortex-A76, 2,4 GHz)	AMD Ryzen 7 3700X (8 núcleos)
Memória	8 GB	32 GB
Armazenamento	SSD NVMe de 256 GB	–
Sistema	Debian 13 (Raspberry Pi OS), *kernel* 6.18.60 (aarch64)	Ubuntu 24.04.5, *kernel* 7.0.0
Alimentação e arrefecimento	Fonte oficial de 27 W; arrefecimento ativo oficial	–
Rede	Ethernet 1 Gb/s	Ethernet 1 Gb/s
Software	GCC 14.2.0, CMake 3.31.6, Mosquitto 2.0.21, Python 3.13.5, OpenSSL 3.5.9	Python 3.12.3

Relógios. A latência é medida entre máquinas: o sensor escreve na mensagem o instante de envio e o *gateway* subtrai-o do instante de entrega. Cada agente estima o desvio do seu relógio em relação ao do Raspberry Pi por trocas de pedido e resposta no canal de controlo (duas por segundo) e os sensores corrigem o instante que escrevem. Dois relógios de quartzo independentes afastam-se a um ritmo quase constante (nos pilotos, cerca de 13 ppm, ou seja, 0,8 ms por minuto, mais do que as latências a medir), pelo que o desvio não é o da última amostra: ajusta-se uma reta ao quarto das amostras do último minuto com menor tempo de ida e volta e avalia-se no instante presente. Os relógios foram sincronizados por NTP antes da campanha e o NTP foi desligado nas duas máquinas durante ela, porque o NTP altera o ritmo do relógio enquanto corrige um desvio, e essa variação não é uma reta.

Tal como planeado, a avaliação tem dois níveis. No **nível 1**, os *microbenchmarks* medem o tempo de processador da cadeia de camadas por mensagem, sem rede, para 16 B, 256 B, 4 KiB e 64 KiB. No **nível 2**, o *broker* e o *gateway* correm no Raspberry Pi e os sensores e o atacante noutras máquinas da rede local.

3.2 Configurações

Foram medidas as doze configurações da Parte 1, Tabela das configurações (K0 a K11), sem alterações. O nível 1 tem um perfil para K0, K2 a K7, K9 e K11; K1, K8 e K10 só diferem das vizinhas no transporte e nos limites, que só existem no nível 2.

3.3 Métricas

A Tabela 3 indica como cada métrica do plano foi medida.

A energia, opcional no plano, não foi medida: não havia um medidor de potência entre a fonte e o Raspberry Pi.

3.4 Ataques

O atacante do laboratório executa os ataques da Parte 1, Tabela dos ataques. Dois tipos de ensaio usam-nos de forma diferente:

Tabela 3. Como cada métrica foi medida.

Métrica	Medição
Processador por mensagem	Tempo de processador do processo do *gateway* na janela, dividido pelas mensagens recebidas (nível 2); tempo de processador por mensagem da cadeia completa (nível 1)
Latência p50 e p99	Do instante de envio, escrito pelo sensor na mensagem, à entrega depois da cadeia de camadas. O relógio de cada máquina de sensores é corrigido pelo desvio estimado em relação ao do *gateway*
Débito máximo sustentado	Maior taxa entregue com perda inferior a 1 % e p99 dentro de $L_{\max}$, numa subida de carga por configuração
Octetos por mensagem	Octetos recebidos na ligação TCP do *gateway* por mensagem, separados em conteúdo, envelope, MQTT e restante (TLS e controlo)
Memória	Pico de memória residente do *gateway* e do *broker* na janela
Estabelecimento de ligação	Tempo e tempo de processador de religações do *gateway* (TLS, CONNECT e subscrições), fora da janela de tráfego
Ataques impedidos, $\pi_a(c)$	Cada ataque contra cada configuração, com tráfego legítimo ativo; maioria sobre as repetições
Fuga de informação	Fração das mensagens legíveis por um subscritor de `#` e por um atacante na ligação do *gateway*

- A **matriz de ataques** lança cada ataque contra cada configuração completa e dá $\pi_a(c)$, de que resulta $S(c)$. Um ataque travado no *broker* (por exemplo, um estranho sem credenciais perante a autenticação mútua) conta como impedido por essa configuração.
- Os **cenários** isolam uma defesa de cada vez sobre uma base aberta e emparelham cada ataque com um *controlo negativo*, a mesma configuração sem essa defesa, em que o ataque tem de ter sucesso. Só assim se atribui uma rejeição à camada certa.

Os ataques internos partem de um dispositivo comprometido: o atacante recebe as chaves, a palavra-passe e o certificado de um sensor, e o sensor verdadeiro fica desligado enquanto o atacante o controla. Os ataques na rede usam um intermediário na ligação entre o *broker* e o *gateway*, que conta o que consegue ler e, em MQTT sem TLS, altera, redireciona, atrasa ou suprime mensagens, fazendo de *broker* malicioso.

3.5 Procedimento

Cada configuração foi repetida 10 vezes, em ordem aleatória com semente registada, com 3 s de estabilização e 30 s de medição por execução, a 1 000 mensagens/s (50 sensores a 20 mensagens/s cada).

O débito máximo foi medido por uma subida de carga, três vezes por configuração, em ordem aleatória: 10, 20, 50, 100, 125, 150, 200, 300, 400, 500, 600, 700, 800, 1 000, 1 200 e 1 600 mensagens/s por sensor (de 500 a 80 000 mensagens/s no total), 15 s cada, parando no primeiro patamar com perda de pelo menos 1 %. Os patamares são mais densos entre 100 e 800 porque é aí que os pilotos situaram os limites. Depois do patamar que satura, a carga volta ao mínimo e espera-se até o *gateway* receber apenas o que os sensores enviam: sem esta espera, as mensagens retidas na sobrecarga chegavam à configuração seguinte, que as rejeitava (outra chave, outro limite de débito) como se fossem tráfego dela.

A campanha foi precedida de três pilotos (quatro configurações, três repetições, 15 s), que serviram para fixar os patamares de carga e R_{alvo} e para validar o próprio instrumento: o primeiro

revelou latências negativas (a deriva dos relógios, corrigida com a estimação descrita acima) e o segundo a contaminação entre patamares (corrigida com a espera). Os dados dos pilotos não são resultados e não entram nas tabelas. Os intervalos de confiança são de 95 %, por *bootstrap* de percentis sobre as execuções. O custo marginal de uma camada é a diferença de medianas entre as duas configurações que só diferem nela, e a diferença das diferenças mede a interação entre a validade temporal e a anti-repetição e entre o TLS e o envelope. A matriz de ataques correu separada das medições de custo, porque o intermediário na ligação do *gateway* acrescenta latência.

Cada campanha guarda os dados brutos de todas as execuções, a ordem dos ensaios, a semente, a configuração completa e um registo do ambiente (código, plataforma, versões, temperatura e frequência do processador por execução). Todas as tabelas e figuras deste relatório são geradas a partir desses dados por um único comando (Anexo A).

3.6 Parâmetros fixados antes das medições

Os parâmetros do critério de escolha foram fixados antes da campanha: o ficheiro que os contém é copiado para a pasta da campanha antes da primeira medição e o seu resumo criptográfico fica no registo do ambiente.

- **Pesos dos ataques.** Cada linha da tabela de ataques da Parte 1 pesa 1. Duas linhas são medidas por dois ataques cada (a escuta, na rede e no *broker*; a injeção e a adulteração), que pesam 0,5 cada.
- **Pesos do custo.** Processador por mensagem 0,4; latência p99 0,4; débito máximo, octetos por mensagem, memória do *gateway* e estabelecimento de ligação 0,05 cada.
- **Exequibilidade.** O cenário de aplicação é a monitorização industrial: um *gateway* de periferia por área de uma instalação, que agrega 1 000 sensores de condição e vibração a 10 Hz e envia ordens a 100 atuadores, até 10 por segundo cada. Daí $R_{\text{alvo}} = 10\,000$ mensagens/s para a telemetria (os dois casos de uso) e $R_{\text{alvo}} = 1\,000$ mensagens/s para as ordens, com $L_{\max} = 50$ ms, um limite de tempo real não estrito adequado à supervisão e ao controlo. Os pilotos serviram apenas para confirmar que nenhum dos valores coincide com o limite de uma configuração, onde o veredicto mudaria de uma repetição para outra.

3.7 Desvios ao plano

- **Aceleração AES desligada.** O plano previa `OPENSSL_armcap=0`. Essa variável desliga todas as extensões ARM que o OpenSSL usa, incluindo o NEON, de que o ChaCha20-Poly1305 depende. Um dispositivo sem as extensões criptográficas tem, em geral, NEON; por isso mediu-se também com `OPENSSL_armcap=1` (só NEON), e é essa a comparação usada para a QP4. Os dois resultados são apresentados.
- **Estabelecimento de ligação.** O tempo de ligação medido varia em degraus de cerca de 100 ms, por esperas internas da biblioteca cliente, que escondem os poucos milissegundos do TLS. No custo agregado entra o tempo de processador por ligação; o tempo decorrido é reportado à parte.

- **Escuta no *broker*.** Na matriz de ataques, este ataque é um subscritor de `#` sem credenciais. Um *broker* curioso não é medido por $S(c)$ (Secção 9); é o requisito de confidencialidade do caso de uso que o cobre.
- **Dispositivo comprometido.** O sensor verdadeiro fica desligado durante o ataque. Com os dois ligados, disputam a sessão MQTT e o ataque fica reduzido a uma fração do tempo (Secção 5.3).
- **R_{alvo} por caso de uso.** O plano previa um só R_{alvo}. As ordens a atuadores são um fluxo muito menor do que a telemetria e só o modo de assinatura (K9) dá a prova de origem que exigem; com um só valor de 10 000 mensagens/s, que K9 não sustenta, esse caso de uso ficaria sem nenhuma configuração exequível por causa de um débito que as ordens nunca têm. Cada caso de uso tem por isso o seu R_{alvo}, fixado antes da campanha como os restantes parâmetros.
- **Instrumento corrigido depois dos pilotos.** A estimação do desvio dos relógios passou a corrigir a deriva e a subida de carga passou a esperar pelo escoamento das mensagens retidas (Secção 3, Procedimento). Não muda o que se mede, só a sua exatidão.
- **Campanhas repetidas.** A campanha completa correu três vezes no mesmo Raspberry Pi, e só a terceira é usada. A primeira não fez a subida de carga (um defeito no *script* de automação descartava os argumentos dela) e mostrou um veredicto instável na ocupação de ligações, que vinha da forma de contar as ligações retidas (contavam-se também as acabadas de abrir, que o *broker* ainda não tinha fechado). A segunda revelou que, com os sensores noutra máquina, o atacante interno não recebia o certificado do dispositivo comprometido: a chave privada de cada sensor é gerada no agente e nunca sai dele, pelo que, com autenticação mútua, o atacante não chegava a ligar-se e os ataques internos contavam como impedidos. O laboratório passou a emitir ao atacante um certificado com o mesmo nome, da mesma autoridade (para o *broker*, a mesma identidade), revogado com o dispositivo. Corrigidos os três defeitos, com testes, a campanha foi repetida por inteiro, para que todos os resultados venham do mesmo código. As medições de custo das três campanhas coincidem dentro do ruído; os veredictos da matriz de ataques das duas primeiras não são válidos e não são usados.
- **Calibração com a subida de carga.** A calibração de Δ_a usa as execuções de custo e só os patamares da subida de carga que a configuração sustenta (perda inferior a 1 % e p99 dentro de $L_{\max}$): num patamar em sobrecarga, a latência de segundos é fila de espera, não entrega legítima.
- **Energia.** Não foi medida (ver acima).

4 Resultados: custo

4.1 Custo por configuração (QP1)

As Tabelas 4 e 5 dão, para cada configuração, a mediana e o intervalo de confiança de cada métrica no Raspberry Pi 5.

Tabela 4. Custo por configuração (processador, latência): mediana e IC 95 % por *bootstrap* sobre as execuções (—).

Config.	CPU (µs/msg)	p50 (µs)	p99 (µs)
K0	24.0 [23.6, 24.4]	121.5 [115.0, 129.5]	173.0 [168.0, 193.0]
K1	31.9 [31.8, 32.1]	149.0 [147.0, 150.5]	220.0 [211.0, 229.0]
K2	39.2 [38.9, 39.4]	178.5 [175.0, 180.5]	260.5 [257.5, 263.0]
K3	39.1 [38.7, 39.6]	174.0 [171.5, 180.0]	262.0 [258.0, 267.5]
K4	39.7 [39.2, 40.0]	176.0 [174.5, 181.0]	257.0 [255.0, 265.0]
K5	39.3 [39.0, 39.9]	177.0 [173.0, 192.0]	258.0 [254.0, 277.5]
K6	39.4 [39.0, 39.9]	184.5 [175.0, 199.5]	268.0 [260.0, 297.0]
K7	40.7 [40.5, 40.9]	179.5 [176.5, 184.5]	260.5 [254.0, 276.5]
K8	41.1 [40.7, 41.4]	179.0 [176.0, 182.0]	262.0 [257.5, 276.0]
K9	192.6 [191.6, 193.0]	379.0 [372.0, 417.0]	725.5 [720.0, 764.0]
K10	35.2 [35.1, 35.3]	151.0 [145.0, 155.5]	224.0 [216.0, 229.0]
K11	42.6 [42.2, 43.0]	178.0 [177.0, 184.0]	268.5 [266.0, 274.0]

Tabela 5. Custo por configuração (memória, octetos, ligação): mediana e IC 95 % por *bootstrap* sobre as execuções (—).

Config.	Memória (KiB)	Octetos/msg	Ligação (ms)
K0	39 456 [39 352, 39 472]	101.8 [101.8, 101.9]	210.7 [210.7, 210.7]
K1	39 480 [39 456, 39 488]	123.8 [123.8, 123.9]	216.2 [216.1, 216.2]
K2	39 464 [39 416, 39 488]	177.9 [177.9, 177.9]	216.2 [216.1, 216.2]
K3	39 464 [39 456, 39 480]	177.8 [177.8, 177.9]	216.2 [216.1, 216.2]
K4	39 456 [39 416, 39 472]	177.9 [177.8, 177.9]	216.1 [216.0, 216.1]
K5	39 472 [39 352, 39 472]	177.9 [177.8, 177.9]	216.1 [216.0, 216.2]
K6	39 416 [37 944, 39 472]	177.9 [177.8, 177.9]	216.1 [216.0, 216.2]
K7	39 472 [39 464, 39 472]	177.9 [177.9, 177.9]	216.1 [216.1, 216.2]
K8	39 456 [39 416, 39 472]	177.9 [177.8, 177.9]	216.1 [216.1, 216.2]
K9	39 456 [31 384, 39 456]	213.9 [213.8, 213.9]	216.1 [216.0, 216.2]
K10	39 456 [39 456, 39 472]	155.8 [155.8, 155.9]	210.7 [210.6, 210.7]
K11	39 456 [39 416, 39 472]	177.8 [177.8, 177.9]	216.1 [216.1, 216.2]

A 1 000 mensagens/s, as configurações formam quatro grupos (Figuras 2 e 3). K0, sem proteção, gasta 24,0 μs de processador do *gateway* por mensagem, com latência p50 de 122 μs e p99 de 173 μs. O transporte sozinho (K1) e o envelope sozinho (K10) ficam a meio: 31,9 e 35,2 μs, p99 de 220 e 224 μs. As configurações com transporte e envelope AES-GCM (K2 a K8 e K11) formam um só grupo, entre 39,1 e 42,6 μs e com p99 entre 257 e 269 μs; dentro dele, as diferenças são de 1 a 3 μs e quase todas cabem no ruído. K9, com assinaturas Ed25519, está isolada: 192,6 μs por mensagem, oito vezes K0, e p99 de 726 μs. Mesmo em K9, a latência continua abaixo de 1 ms, cinquenta vezes menos do que $L_{\max}$; à carga nominal, o custo de todas as configurações é pequeno em termos absolutos, e o que as separa é o débito máximo (Secção 4.3).

4.2 Custo marginal de cada camada

A Tabela 6 dá o custo de cada camada isolada:

- **Transporte** (K0→K1): +7,9 μs por mensagem e +47 μs no p99, mais 22 octetos por mensagem na ligação do *gateway* e 4,8 MB de memória no *broker*.
- **Envelope** (K1→K2): +7,3 μs e +41 μs no p99, quase tanto como o TLS, e 53 octetos

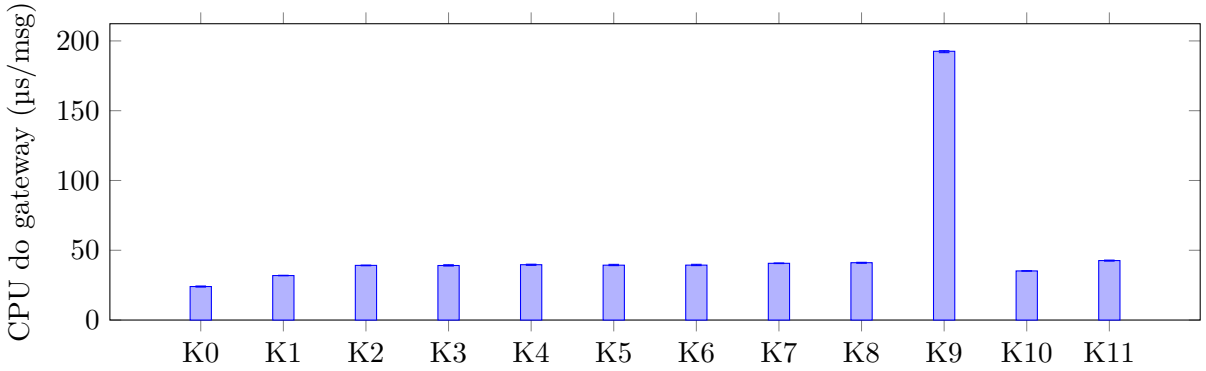


Figura 2. Tempo de processador do *gateway* por mensagem, por configuração (mediana e IC 95 %).

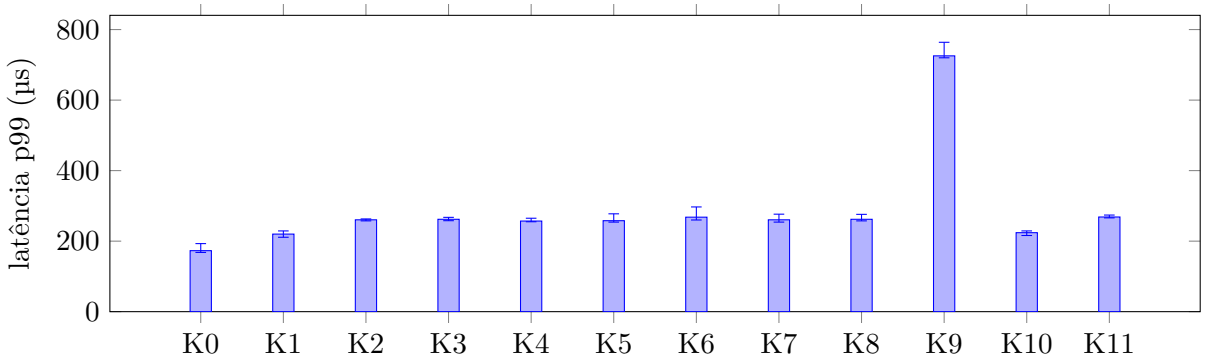


Figura 3. Latência p99 por configuração (mediana e IC 95 %).

por mensagem. Passar de GMAC a AES-256-GCM (K2→K3) não custa nada mensurável ($-0{,}1$ μs, IC $[-0{,}5; 0{,}5]$): com AES em *hardware*, cifrar 16 octetos é desprezável face ao resto do envelope (análise, procura da chave, verificações).

- **Validade temporal** (K3→K4) e **anti-repetição** (K3→K5): diferenças de 0,6 e 0,2 μs, com intervalos que incluem zero.
- **Restrição chave–tópico** (K6→K7): $+1{,}3$ μs (IC $[0{,}7; 1{,}8]$), a única verificação barata que se distingue do ruído.
- **Limitação de tráfego e limites do *broker*** (K7→K8): $+0{,}4$ μs, dentro do ruído.
- **Assinatura em vez de AES-GCM** (K8→K9): $+151{,}5$ μs por mensagem, $+464$ μs no p99 e $+36$ octetos (89 em vez de 53).
- **ChaCha20-Poly1305 em vez de AES-256-GCM** (K8→K11): $+1{,}5$ μs, num processador com AES em *hardware* (Secção 4.4).

Interações. A validade e a anti-repetição custam o mesmo juntas e separadas: a diferença das diferenças é de $-0{,}5$ μs, IC $[-1{,}3; 0{,}4]$. O TLS custa menos com o envelope ativo do que sem ele (5,5 contra 7,9 μs, uma interação de $-2{,}4$ μs, IC $[-2{,}9; -2{,}0]$); a causa não foi investigada. Para a latência, nenhuma das interações se distingue do ruído.

Tabela 6. Custo marginal de cada camada (diferença de medianas, IC 95 %).

Passo	Camada	CPU (µs/msg)	p99 (µs)
K0→K1	transporte (mTLS 1.3, autenticação, ACL)	7.9 [7.5, 8.3]	47.0 [23.0, 57.5]
K1→K2	envelope de autenticação (GMAC)	7.3 [6.9, 7.5]	40.5 [31.0, 50.0]
K2→K3	cifra (AES-256-GCM em vez de GMAC)	-0.1 [-0.5, 0.5]	1.5 [-3.5, 7.0]
K3→K4	validade temporal	0.6 [-0.1, 1.1]	-5.0 [-10.0, 3.5]
K3→K5	janela anti-repetição	0.2 [-0.4, 0.9]	-4.0 [-9.5, 15.5]
K6→K7	restrição chave–tópico	1.3 [0.7, 1.8]	-7.5 [-36.5, 9.5]
K7→K8	limitação de tráfego e limites do broker	0.4 [-0.1, 0.8]	1.5 [-13.0, 16.5]
K8→K9	assinatura Ed25519 em vez de AES-GCM	151.5 [150.5, 152.3]	463.5 [449.0, 499.0]
K10→K7	TLS com o envelope já ativo	5.5 [5.2, 5.7]	36.5 [26.5, 52.5]
K8→K11	ChaCha20-Poly1305 em vez de AES-256-GCM	1.5 [1.1, 2.1]	6.5 [-7.5, 13.5]

4.3 Tamanho da mensagem e saturação (QP3)

Tamanho. No nível 1 (Tabela 8), sem rede, o custo de K0 quase não depende do tamanho (0,8 μs a 16 B, 7,1 μs a 64 KiB). Com AES em *hardware*, o envelope AES-GCM (K3) custa 5,6 μs a 16 B, 6,0 a 256 B, 13,0 a 4 KiB e 118 μs a 64 KiB: até alguns KiB, domina a parte fixa do envelope, e só acima disso o custo passa a crescer com o tamanho. A assinatura (K9) é dominada pela parte fixa em toda a gama (266 μs a 16 B, 301 μs a 4 KiB, 820 μs a 64 KiB). No nível 1 cada mensagem é selada e aberta, pelo que estes valores incluem o emissor e o recetor; no nível 2 o *gateway* só verifica, e a diferença K8→K9 é de 151 μs. O tamanho não foi variado no nível 2.

Saturação. A Tabela 7 dá o débito máximo sustentado (perda inferior a 1 % e p99 até $L_{\max}$) e o processador no último patamar sustentado. Em todas as configurações, o que limita primeiro é o processador do *gateway*: no patamar seguinte, o seu processo chega a 93–100 % de um núcleo (a cadeia de camadas corre numa só linha de execução), enquanto o *broker* fica entre 15 e 80 %. K0 sustenta 60 000 mensagens/s, o transporte desce para 40 000, as configurações com envelope AES ficam entre 25 000 e 35 000 e K9 fica em 5 000 mensagens/s, metade de R_{alvo} da telemetria. Os patamares são discretos (passos de 5 000 mensagens/s entre 400 e 800 por sensor): uma diferença de um patamar entre K3 e K8 está dentro da resolução da subida. K6, K7 e K10 sustentaram 30 000 mensagens/s numa das três subidas e falharam nas outras duas, com p99 de 120 a 520 ms; a mediana fica no patamar anterior.

A partir de 400 mensagens/s por sensor (20 000 mensagens/s), a latência sobe de forma brusca em todas as configurações, incluindo K0: o p99 passa de 0,2–0,6 ms para 20–35 ms e depois volta a descer com a carga, sem perda. Isto acontece com o *gateway* a 40 % (K0) ou a 63 % (K8) de processador, e K9 satura antes de lá chegar. O efeito foi registado, mas a sua origem não foi determinada (Secção 9). Fica abaixo de $L_{\max}$ e acima de R_{alvo} (200 mensagens/s por sensor), pelo que não muda nenhum veredicto.

Tabela 7. Débito máximo sustentado no Raspberry Pi 5 (mediana das três subidas) e processador do *gateway* e do *broker* no último patamar sustentado.

Config.	Débito (msg/s)	*Gateway* (%)	*Broker* (%)
K0	60 000	95	56
K1	40 000	93	72
K2	35 000	96	61
K3, K4, K5	30 000	84–87	55–57
K6, K7	25 000	74–76	44–48
K8	30 000	89	55
K9	5 000	93	14
K10	25 000	70	45
K11	30 000	92	57

4.4 Aceleração criptográfica por *hardware* (QP4)

O Cortex-A76 do Raspberry Pi 5 tem as extensões criptográficas ARMv8, e o OpenSSL usa-as para o AES e o GHASH. Sem elas, só com NEON (`OPENSSL_armcap=1`), o envelope AES-GCM (K3) passa de 5,6 para 7,5 μs a 16 B, de 6,0 para 12,4 a 256 B e de 13,0 para 88 μs a 4 KiB, sete vezes mais. O ChaCha20-Poly1305 (K11) quase não depende das extensões (6,9 a 7,6 μs a 16 B; 21,7 a 22,4 μs a 4 KiB), porque usa NEON; só sem NEON (`OPENSSL_armcap=0`) chega a 38 μs a 4 KiB.

Daqui resulta a regra de escolha da Parte 1, agora com números. Com AES em *hardware*, o AES-GCM é o mais barato em todos os tamanhos (13,0 contra 21,7 μs a 4 KiB), e no nível 2 K11 custa mais 1,5 μs por mensagem do que K8. Sem AES em *hardware*, o ChaCha20 compensa a partir de algumas centenas de octetos: a 256 B já custa 8,7 contra 12,4 μs, e a 4 KiB custa quatro vezes menos. A 16 B, a diferença é inferior a 0,1 μs e quem decide é o resto da cadeia. A assinatura não muda com as extensões, porque o Ed25519 não as usa.

4.5 Octetos e memória

O envelope acrescenta 53 octetos por mensagem com AES-GCM ou ChaCha20-Poly1305 e 89 com Ed25519, exatamente o que o formato prevê; com o conteúdo de 16 octetos usado na campanha, a mensagem na ligação do *gateway* passa de 102 octetos (K0) para 178 (K2 a K8, K11) e 214 (K9). O TLS acrescenta 22 octetos por mensagem, a que se soma a negociação de cada ligação, que custa 4,2 ms de processador no *gateway* com autenticação mútua, contra 0,8 ms sem TLS. A memória do *gateway* é a mesma em todas as configurações (cerca de 39 MB, um majorante, Secção 9); a do *broker* passa de 7,5 MB sem TLS para cerca de 12,3 MB com TLS, e não muda com as restantes camadas.

4.6 Plataforma de referência

[**Por fazer:** O mesmo no computador de secretária (Anexo A): em que é que a ordenação das configurações muda, se mudar.]

5 Resultados: proteção

Tabela 8. Nível 1: tempo de processador por mensagem (µs, mediana das repetições), por configuração, tamanho e aceleração AES (`OPENSSL_armcap=0` desliga todas as extensões ARM; `OPENSSL_armcap=1` mantém o NEON).

Config.	AES-HW	16 B	256 B	4096 B	65536 B
K0	sim	0.82	0.84	1.30	7.09
K0	não (NEON)	0.81	0.83	1.41	7.11
K0	não	0.81	0.84	1.29	7.24
K2	sim	5.63	5.90	10.83	85.02
K2	não (NEON)	7.43	9.94	49.93	684.37
K2	não	7.22	9.72	49.61	683.77
K3	sim	5.59	6.04	13.00	118.30
K3	não (NEON)	7.53	12.41	88.12	1 287
K3	não	7.32	12.93	101.92	1 521
K4	sim	5.70	6.13	13.03	118.27
K4	não (NEON)	7.62	12.52	88.01	1 288
K4	não	7.44	13.03	102.03	1 522
K5	sim	5.65	6.12	13.01	118.10
K5	não (NEON)	7.60	12.48	87.91	1 288
K5	não	7.39	12.99	101.99	1 521
K6	sim	5.75	6.21	13.09	118.78
K6	não (NEON)	7.71	12.57	88.00	1 288
K6	não	7.50	13.09	102.08	1 521
K7	sim	6.13	6.58	13.43	119.11
K7	não (NEON)	8.06	12.94	88.28	1 288
K7	não	7.86	13.45	102.48	1 522
K9	sim	265.84	268.02	300.84	819.60
K9	não (NEON)	266.00	268.18	301.01	818.54
K9	não	266.19	268.38	301.18	820.37
K11	sim	6.95	8.05	21.66	239.33
K11	não (NEON)	7.61	8.69	22.37	241.58
K11	não	7.42	8.96	38.10	500.14

5.1 Ataques impedidos por configuração (QP2)

A Tabela 9 dá $\pi_a(c)$ para os treze ataques da matriz e as doze configurações, cada célula decidida por maioria de três repetições, e $S(c)$ com os pesos da Secção 3.6. As três repetições concordaram em todas as 156 células.

A leitura por camadas é a seguinte:

- **O transporte (K1) impede tudo o que exige chegar ao *broker* sem credenciais**: escuta na rede e no *broker*, injeção, adulteração e republicação no caminho, repetição por um estranho e usurpação da identidade do *gateway*. Pela ACL, impede também que um dispositivo comprometido publique no tópico de outro (A6) ou forje ordens para outro atuador (A10). Não impede o que um dispositivo comprometido faz no seu próprio tópico: mensagens antigas (A8), despromoção do modo (A9), inundação (A12), nem a ocupação de ligações (A13). $S = 0{,}64$.

- **O envelope sozinho (K10)**, sem TLS nem ACL, impede quase o mesmo por outro caminho: o conteúdo é ilegível e a autenticação, o tópico nos dados associados, a restrição chave–tópico, a validade e a janela tornam inúteis as mensagens forjadas, alteradas, republicadas ou repetidas, mesmo quando chegam ao *gateway*. Por não ter ACL nem autenticação no *broker*, não impede

Tabela 9. Ataques impedidos, $\pi_a(c)$ (✓ impedido; maioria de 3 repetições; as repetições concordaram em todas as células), e proteção $S(c)$. A1 eavesdrop-network; A2 eavesdrop-broker; A3 inject; A4 tamper-on-path; A5 republish-topic; A6 stolen-key-other-topic; A7 replay; A8 old-unseen-message; A9 downgrade; A10 receiver-forgery; A11 impersonate; A12 flood; A13 slow-connections.

Config.	A1	A2	A3	A4	A5	A6	A7	A8	A9	A10	A11	A12	A13	$S(c)$
K0	–	–	–	–	–	–	–	–	–	–	–	–	–	0.00
K1	✓	✓	✓	✓	✓	✓	✓	–	–	✓	✓	–	–	0.64
K2	✓	✓	✓	✓	✓	✓	✓	–	–	✓	✓	–	–	0.64
K3	✓	✓	✓	✓	✓	✓	✓	–	✓	✓	✓	–	–	0.73
K4	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	–	–	0.82
K5	✓	✓	✓	✓	✓	✓	✓	–	✓	✓	✓	–	–	0.73
K6	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	–	–	0.82
K7	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	–	–	0.82
K8	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	1.00
K9	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	1.00
K10	✓	✓	✓	✓	✓	✓	✓	✓	✓	–	–	–	–	0.64
K11	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	1.00

a usurpação do *gateway* (A11) nem a falsificação de ordens por um atuador, que tem a chave partilhada (A10). $S = 0{,}64$, o mesmo valor que K1, com ataques diferentes.

- **Só a combinação impede os ataques de um dispositivo comprometido no seu próprio tópico.** A política de modo do recetor (a partir de K3) impede a despromoção para autenticação sem cifra. A validade temporal (K4, K6 e seguintes) impede a mensagem antiga nunca recebida, que a janela anti-repetição sozinha (K5) aceita, porque os seus números de sequência são novos: K5 tem por isso menos proteção do que K4 (0,73 contra 0,82).

- **Só K8 e seguintes impedem a inundação e a ocupação de ligações**: a limitação de tráfego descarta cerca de 1 560 das 1 810 mensagens da inundação antes da criptografia, e o limite de ligações do *broker* deixa o atacante com 15 ligações em vez das 200 que tenta reter.

K8, K9 e K11 impedem todos os ataques da matriz ($S = 1$). K9 não acrescenta nada a K8 na matriz: a prova de origem que a assinatura dá só se vê quando o *broker* não aplica a ACL, o que a matriz não mede (Secção 5.3).

5.2 Cada ataque contra a sua defesa, com controlo negativo

A Tabela 10 resume os cenários: cada linha isola uma defesa sobre uma base aberta e mostra o mesmo ataque sem ela (controlo) e com ela. Os 39 cenários deram o resultado previsto no Raspberry Pi, na mesma campanha.

Tabela 10. Cenários: cada ataque contra a defesa que o deve impedir, com controlo negativo (Raspberry Pi 5; 6 s por cenário, cerca de 120 mensagens de ataque).

Ataque	Defesa isolada	Sem a defesa	Com a defesa
Subscrever **#**	ACL, zona restrita	100 % das leituras	0 %, tráfego legítimo intacto
Escutar a ligação do *gateway*	TLS	100 % legível	nada legível na ligação, mas um subscritor de **#** lê tudo

Ataque	Defesa isolada	Sem a defesa	Com a defesa
Subscrever `#` e escutar	envelope em modo de cifra, sem TLS	–	100 % das mensagens recebidas, 0 % legíveis
Usurpar `gateway`	autenticação por palavra-passe	ligado como `gateway`, lê tudo	recusado pelo *broker*
Injetar leituras	GMAC; Ed25519; ChaCha20	121 aceites	0 aceites em cada modo
Republicar noutro tópico	tópico nos dados associados	117 de 121 aceites	0; 121 falham a autenticação
Chave roubada noutro tópico	restrição chave–tópico; ACL	120 de 121 aceites	0 (121 `PermissionDenied`); 0 com ACL
Mensagem antiga nunca vista	validade ($\Delta_a = 60$ s)	120 aceites (só a janela)	0 (120 `MessageExpired`)
Despromover o modo	modo nos dados associados; política do recetor	120 aceites (política aceita os dois)	0 em ambos
Atuador forja ordens	assinatura	116 de 121 aceites (chave AES partilhada)	0; as ordens verdadeiras continuam aceites
Repetir mensagens	janela anti-repetição	120 aceites (só cifra; e com validade de 60 s)	0 (120 `ReplayDetected`)
Inundar o próprio tópico	limite de 30 msg/s por tópico	–	902 descartadas antes da criptografia; recuperação em 0,8 s
Reter 300 ligações	limite de ligações	300 retidas	14 retidas, sensores ligados, novos clientes recusados; recuperação em 0,7 s
Broker altera ou redireciona	envelope	3 013 alteradas, 0 rejeitadas	cerca de 3 010 rejeitadas em cada caso
Broker atrasa 2 s	validade ($\Delta_a = 60$ s)	–	não impedido: 0 rejeitadas, p99 de 2,0 s
Broker suprime metade	envelope	–	não impedido: 50 % de perda, sem rejeição

A tabela confirma, linha a linha, a tabela de ataques da Parte 1, incluindo as duas linhas que ela dava como não impedidas: o atraso abaixo de Δ_a e a supressão por um *broker* ativo. Os restantes cenários verificam que as defesas não estragam o tráfego legítimo: as ordens do *gateway* protegidas fim-a-fim (120 de 121 aceites, nenhuma rejeitada), TLS 1.3 com MQTT 5, todas as defesas ao mesmo tempo, a rotação e a revogação (Secção 8).

5.3 Observações

Os ensaios mostraram efeitos que a tabela de ataques da Parte 1 não previa. Os dois primeiros foram medidos na campanha; os três seguintes foram observados durante o desenvolvimento e são evitados pelo desenho dos ensaios (Secção 3.7).

- **Falsificação pelo recetor num *broker* legítimo.** A ACL já impede um atuador de publicar

nos tópicos de ordens dos outros: na matriz, K1 a K8 impedem este ataque tal como K9. A diferença entre a chave partilhada e a assinatura só se observa sem ACL (116 de 121 ordens forjadas aceites com AES, nenhuma com Ed25519) ou com um *broker* ativo.

- **Atraso e supressão.** Um *broker* ativo que atrase mensagens 2 s (menos do que Δ_a) ou que suprima metade delas não é detetado pelo recetor: só se vê como latência ou como perda.
- **Ocupação de ligações.** O limite de ligações protege os recursos do *broker* e mantém os clientes já ligados, mas o atacante ocupa as vagas livres e os clientes novos são recusados enquanto o ataque dura. Se o *broker* reiniciar durante o ataque, o atacante pode ficar com todas as vagas e nenhum sensor volta a ligar-se; por isso, no cenário, o limite está em vigor antes do ataque.
- **Chave roubada e janela da vítima.** As mensagens aceites com uma chave roubada avançam a janela anti-repetição dessa chave; o tráfego legítimo do dispositivo passa a ser rejeitado até à rotação de chaves.
- **Credencial clonada e sessão MQTT.** Se o atacante se liga com a identidade de um dispositivo que continua ligado, o *broker* entrega-lhe a sessão e desliga o legítimo; os dois alternam enquanto o ataque dura, o que é uma negação de serviço da vítima.

6 Critério de escolha e recomendação

Aplica-se o critério da Parte 1, Secção 7.5 com os parâmetros da Secção 3.6: proteção $S(c)$, custo normalizado por K0, exequibilidade, requisitos por caso de uso e fronteira de Pareto.

Tabela 11. Proteção $S(c)$, custo custo(c), exequibilidade e fronteira de Pareto por caso de uso (T telemetria não sensível, TS telemetria sensível, O ordens; * configuração recomendada c^*). R_{alvo} por caso de uso: T 10 000, TS 10 000, O 1 000 mensagens/s; $L_{\max} = 50$ ms.

Config.	$S(c)$	Custo	Exequível	Pareto
K0	0.00	1.00	T TS O	–
K1	0.64	1.50	T TS O	–
K2	0.64	1.75	T TS O	–
K3	0.73	1.77	T TS O	–
K4	0.82	1.76	T TS O	–
K5	0.73	1.76	T TS O	T TS
K6	0.82	1.80	T TS O	–
K7	0.82	1.81	T TS O	–
K8	1.00	1.80	T TS O	**T* TS***
K9	1.00	5.91	O	**O***
K10	0.64	1.40	T TS O	T TS
K11	1.00	1.84	T TS O	–

Os requisitos de cada caso de uso (Parte 1, Secção 7.5) excluem logo K0 a K4: a telemetria exige integridade, ligação ao tópico e anti-repetição no conteúdo, e as ordens exigem ainda validade, restrição chave–tópico e prova de origem.

- **Telemetria não sensível e telemetria sensível.** As configurações admissíveis são as mesmas nos dois casos (K5 a K8, K10 e K11; a confidencialidade exigida pela telemetria

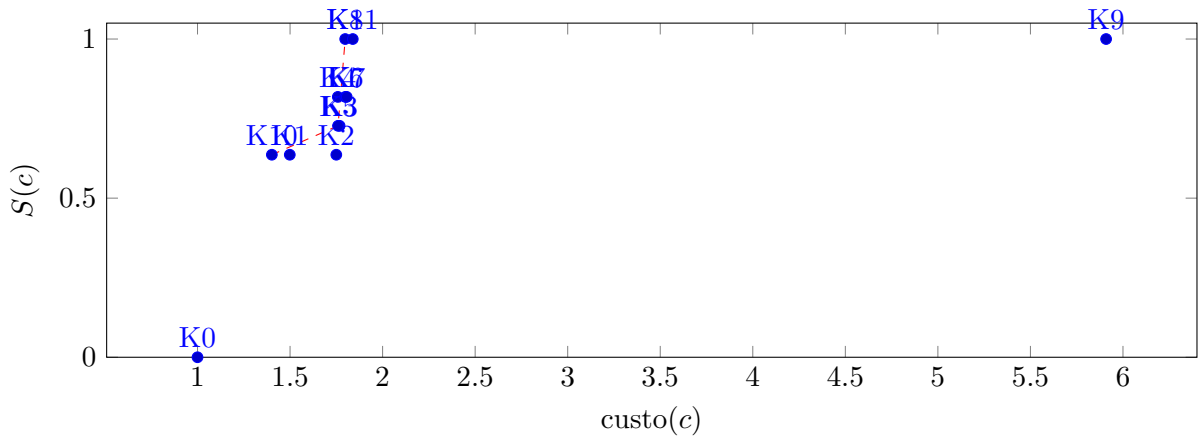


Figura 4. Proteção $S(c)$ contra custo(c); a tracejado, a fronteira de Pareto para a telemetria sensível.

sensível é dada pelo modo de cifra, que todas usam). K9 cumpre os requisitos da telemetria não sensível mas não é exequível: sustenta 5 000 mensagens/s, metade de R_{alvo}. A fronteira de Pareto tem três pontos: K10 ($S = 0{,}64$, custo 1,40), K5 (0,73; 1,76) e K8 (1,00; 1,80). O custo marginal ao longo da fronteira é de 3,9 por unidade de S de K10 para K5 e de apenas 0,14 de K5 para K8: os últimos 27 pontos de proteção (validade, limitação de tráfego e limites do *broker*) quase não custam. $c^* =$ K8 nos dois casos.

- **Ordens a atuadores.** Só K9 dá prova de origem, e é exequível para as ordens ($R_{\text{alvo}} =$ 1 000 mensagens/s, que sustenta cinco vezes, com p99 de 0,7 ms à carga nominal). $c^* =$ K9, sem alternativa: o seu custo, 5,9 vezes o de K0, é o preço da prova de origem.

A recomendação antecipada na Parte 1 mantém-se: **K8 para a telemetria e K9 para as ordens**. K11 (ChaCha20) tem a mesma proteção que K8 e custa mais 2 % neste processador, que tem AES em *hardware*; num processador sem essas extensões, o envelope AES custa 1,4 vezes o do ChaCha20 a 256 octetos e 4 a 5 vezes a partir de 4 KiB (Secção 4.4) e K11 passa a ser a escolha para a telemetria, como a Parte 1 previa.

Sensibilidade. A escolha não depende dos pesos fixados. Com todos os ataques a pesar 1, com pesos de custo iguais, ou com todo o peso no processador ou na latência p99, c^* continua a ser K8 para a telemetria e K9 para as ordens. Só com todo o peso nos octetos por mensagem ou no débito máximo é que K8 e K11 empatam, porque têm o mesmo envelope e saturam no mesmo patamar. A ordem das configurações com $S = 1$ é sempre a mesma, porque K8 não é pior do que K11 em nenhuma métrica de custo, e K9 é a mais cara em processador, latência, octetos e débito.

7 Hipóteses

A evidência de cada hipótese está em `gerado/pi5/hypotheses.md`; o veredicto é dos autores.

H1 (confirmada, com ressalva). Em valor absoluto, o envelope custa pouco: 7,2 μs de processador e 25 μs de latência mediana, mil vezes menos do que $L_{\max}$ e menos do que a latência

Tabela 12. Veredicto das hipóteses da Parte 1.

Hipótese	Evidência	Veredicto
H1. O envelope AES-256-GCM tem um custo por mensagem pequeno face à latência da rede	K1→K3: +7,2 μs de processador (IC [6,8; 7,8]) e +25 μs no p50 (149 μs em K1); débito máximo de 40 000 para 30 000 mensagens/s	Confirmada, com ressalva
H2. Validade, anti-repetição e chave–tópico têm custo marginal quase nulo	K3→K4 +0,6 μs e K3→K5 +0,2 μs (IC incluem 0); K6→K7 +1,3 μs (3 %); p99 sem diferença mensurável	Confirmada
H3. Ed25519 custa claramente mais do que AES-GCM por mensagem	K8→K9: +151,5 μs (4,7 vezes), +464 μs no p99, débito máximo seis vezes menor (5 000 contra 30 000 mensagens/s)	Confirmada
H4. O custo do TLS concentra-se no estabelecimento da ligação	Por ligação: 4,2 contra 0,8 ms de processador. Por mensagem: K0→K1 +7,9 μs e K10→K7 +5,5 μs; débito máximo de 60 000 para 40 000 mensagens/s	Refutada
H5. Sob inundação, a limitação antes da criptografia reduz o processador do *gateway*	Inundação a 300 msg/s: K7 5,2 %, K8 5,1 % de processador (40,5 e 39,7 μs por mensagem); K8 descarta 1 559 mensagens antes da criptografia	Não confirmada

de qualquer rede sem fios típica da IoT. Mas, nesta rede local cablada, a latência é tão baixa que o envelope representa 17 % do p50 de K1, e reduz o débito máximo em 25 %. A hipótese vale para a latência; para o débito, o envelope não é desprezável.

H4 (refutada). A negociação TLS com autenticação mútua custa 3,4 ms de processador a mais por ligação, mas o TLS custa também 7,9 μs por mensagem, tanto como o envelope inteiro (7,2 μs), e é o que mais reduz o débito máximo (de 60 000 para 40 000 mensagens/s). Ao fim de cerca de 430 mensagens, o custo por mensagem ultrapassa o da negociação; a 1 000 mensagens/s, isso acontece em menos de meio segundo, e um sensor a 10 Hz chega lá em 43 s. Em ligações duradouras, que são as do MQTT, o custo do TLS está nas mensagens, não no estabelecimento. O custo por mensagem é menor com o envelope ativo (5,5 contra 7,9 μs, Secção 4), mas continua a dominar.

H5 (não confirmada). A limitação funciona como previsto: descarta o excesso antes de qualquer verificação criptográfica (1 559 de 1 814 mensagens da inundação). Mas o processador do *gateway* é o mesmo em K7 e K8 (5,2 e 5,1 %). Com AES em *hardware*, verificar uma mensagem custa poucos microssegundos: o que a limitação poupa, cerca de 260 verificações por segundo, é da ordem de 0,2 % de um núcleo, abaixo do ruído. Com verificações mais caras, como as assinaturas, a poupança poderia ser maior, mas a matriz não tem uma configuração com assinaturas e sem limitação que o meça. A hipótese, tal como formulada para o envelope AES-GCM, não se confirma. O que a limitação mostrou foi a proteção da aplicação: sem ela (K7), as mensagens da inundação são todas entregues; com ela (K8), o excesso é descartado (Secção 5).

8 Validação do protótipo

8.1 Conformidade com a especificação

A validação tem três níveis. Os **testes automatizados** (Secção 2) verificam cada regra de aceitação, os vetores de teste do envelope (partilhados pela implementação em C++ e pela em Python,

que assim não podem divergir) e os vetores do RFC 8439 para o ChaCha20-Poly1305. Os **cenários** (Secção 5) verificam o sistema completo contra um *broker* real: os 39 cenários deram o resultado previsto no Raspberry Pi, na própria campanha (`results/pi5-20261113/scenarios.md`). No mesmo *commit* da campanha (`79c4878`), os 93 testes em C++ e os 61 testes em Python do laboratório passaram no Raspberry Pi (`results/validation/pi5-20261113/`); só os sete testes que dependem do `tshark` não correram, por o Wireshark não estar instalado no Raspberry Pi. A **matriz de ataques** confirma que cada configuração impede o que o desenho prevê (Tabela 9), com as três repetições de acordo em todas as células.

8.2 Validação do instrumento de medição

Os pilotos no Raspberry Pi validaram também o próprio laboratório, e revelaram dois defeitos que teriam falseado a campanha:

- **Deriva dos relógios.** No primeiro piloto, a latência mediana de K0 saiu negativa (-455 μs) e variava de 300 a 1 055 μs entre repetições da mesma configuração. O desvio entre os relógios crescia de forma regular (de 32,9 para 51,8 ms ao longo da subida de carga, cerca de 13 ppm), e a estimação usava a melhor amostra do último minuto, que podia estar 0,8 ms atrasada. Com a reta ajustada às amostras (Secção 3), a latência mediana de K0 ficou entre 132 e 133 μs nas três repetições do terceiro piloto. Um teste reproduz a deriva observada e exige um erro inferior a 50 μs.
- **Contaminação entre patamares.** No segundo piloto, uma execução de K8 no primeiro patamar (500 mensagens/s) registou 77 388 rejeições e p99 de 1,3 s: seguia-se a um patamar de K11 em sobrecarga, cujas mensagens retidas chegaram depois da mudança de chave e de limite de débito. Se a perda tivesse passado de 1 %, a subida de carga de K8 teria parado logo nesse patamar e o seu débito máximo ficaria por medir. Com a espera pelo escoamento, nenhuma execução posterior a uma saturação teve rejeições no terceiro piloto.

8.3 Calibração da validade temporal

A Parte 1 fixou valores iniciais, $\Delta_a = 60$ s e $\Delta_s = 5$ s, a calibrar nesta parte a partir da latência medida e do desvio de relógio observado. Cada execução regista a cauda da latência e o desvio entre o relógio de cada máquina de sensores e o do *gateway*.

Na campanha, a maior latência de uma mensagem legítima foi de 43,2 ms (p99,9 máximo de 37,1 ms), contando as execuções de custo e os patamares da subida de carga que cada configuração sustenta. Com o NTP desligado, o desvio entre os relógios das duas máquinas cresceu até 289 ms ao longo das cinco horas; depois da correção pelos agentes, a incerteza residual nunca passou de 0,21 ms. Com margem de segurança 2 e arredondamento a 100 ms, os valores propostos são:

- **relógios sincronizados** (NTP ou PTP nos dispositivos): $\Delta_s = 100$ ms e $\Delta_a = 100$ ms;
- **relógios sem sincronização**, em que Δ_s tem de cobrir o desvio observado: $\Delta_s = 600$ ms e $\Delta_a = 700$ ms.

Os dois casos estão muito abaixo dos 60 s e 5 s iniciais da Parte 1: uma mensagem antiga só é aceite se tiver menos de um segundo, em vez de um minuto. O segundo caso depende de quanto tempo os relógios ficam sem acerto (aqui, cinco horas a cerca de 16 ppm); num dispositivo sem relógio de tempo real que reinicia, o desvio pode ser arbitrário, e é a sincronização que o resolve, não um Δ_s maior. Para as ordens a atuadores, em que uma ordem atrasada é ela própria um risco, a validade curta é a que interessa: com relógios sincronizados, uma ordem com mais de 100 ms é recusada. Por fluxo (`FreshnessRule`), a telemetria pode manter uma validade mais larga sem afetar as ordens.

8.4 Reinício do recetor

O recetor guarda em disco a última época de chaves que aceitou. Depois de um reinício, a janela anti-repetição está vazia, mas só são aceites chaves de uma época posterior: as mensagens da época anterior, incluindo repetições capturadas antes do reinício, são rejeitadas como repetição, e o tráfego retoma quando os emissores passam à época seguinte. Um teste de aceitação verifica esta sequência (`AfterARestartOnlyALaterEpochIsAccepted`), e outro que uma falha a ler ou a escrever a época recusa as mensagens em vez de as aceitar. No laboratório, o *gateway* corre sempre com esta época em disco. O tempo sem serviço depois de um reinício é, por isso, o tempo que a implantação leva a passar os emissores à época seguinte. Depende da distribuição de chaves, fora do âmbito deste trabalho (Secção 9), e não foi medido.

8.5 Rotação e revogação

Os dois cenários passaram no Raspberry Pi. Em `key-rotation`, todas as chaves do conteúdo são substituídas com o tráfego a correr, sem nenhuma rejeição e com perda de 0,1 %. Em `revocation`, o sensor 1 é revogado (chaves e palavra-passe): fica de fora e os outros 49 continuam, com o débito entregue a descer de 1 000 para 980 mensagens/s, exatamente a parte do sensor revogado.

9 Ameaças à validade e limitações

9.1 Validade de construção

- $S(c)$ **conta ataques, não a sua gravidade.** Com os pesos usados, todos os ataques valem o mesmo; a exclusão por caso de uso compensa em parte, porque uma propriedade obrigatória em falta não é trocada por custo.
- **A escuta no *broker* mede um subscritor estranho.** Na matriz de ataques, quem escuta é um cliente sem credenciais, que K1 já impede. Um *broker* curioso ou comprometido lê o conteúdo em K1, K2 e K9, que não cifram perante o *broker*, e $S(c)$ não o mostra. Quem o mostra é o filtro por caso de uso (a telemetria sensível exige o modo de cifra) e o cenário que compara o que um subscritor lê com TLS e com o envelope.
- **Um ataque travado no *broker* conta como impedido.** As configurações com transporte protegido impedem muitos ataques por o atacante não chegar ao *broker*; isso deixa de valer se

o *broker* for o adversário. K10 e os cenários com *broker* ativo medem o que resta sem essa barreira.

- ***Broker* ativo só em MQTT sem TLS.** O intermediário não termina TLS; um *broker* malicioso real fá-lo-ia. O que o envelope deteta não depende do transporte, pelo que o resultado se mantém, mas não foi medido com TLS.

9.2 Validade interna

- **O instrumento pesa na medição.** O tempo de processador do *gateway* inclui a contabilidade das medições, e a sua memória inclui as amostras de latência da janela e a tabela de mensagens vistas, que cresce nos primeiros minutos de cada laboratório. As diferenças entre configurações são válidas; os valores absolutos de memória são um majorante.
- **Tempo de ligação.** Varia em degraus de cerca de 100 ms por esperas internas da biblioteca cliente; usa-se o tempo de processador por ligação.
- **Relógios.** A latência entre máquinas depende da correção do relógio dos sensores, com incerteza igual a metade do melhor tempo de ida e volta observado, no máximo 0,21 ms na campanha (0,18 ms nas execuções de custo). É da ordem das diferenças de latência mediana entre configurações vizinhas (K0 122 μs, K8 179 μs), mas a mesma para todas: como a ordem é aleatória, desloca as medidas sem favorecer nenhuma configuração. A deriva é corrigida (Secção 3); uma alteração do ritmo do relógio a meio de uma execução não o é, e por isso o NTP esteve desligado durante a campanha. O desvio entre as duas máquinas cresceu de forma regular até 289 ms (cerca de 16 ppm), sempre corrigido.
- **Temperatura e frequência.** Nos três pilotos não houve *throttling* (`throttled` sempre `0x0`), a frequência esteve sempre em 2 400 MHz e a temperatura entre 44 e 52 °C. O mesmo na campanha: `0x0` e 2 400 MHz em todas as execuções, temperatura entre 44,6 e 52,4 °C (o máximo na subida de carga). A ordem aleatória distribui o efeito que houver pelas configurações.
- **Sensores simulados.** Os sensores são processos Python num computador, não dispositivos restritos: o custo no emissor não foi medido no nível 2 (o nível 1 mede envio e receção).
- **Uma só máquina de carga.** Os dois agentes de sensores e o atacante correm no mesmo computador. Ele gerou 80 000 mensagens/s nos pilotos sem ficar aquém da taxa pedida, mas o que acontece nele (escalonamento, recolha de memória do Python) entra na latência medida.
- **Picos de latência.** Em todas as configurações, incluindo K0, que não tem criptografia, surgem nos pilotos picos de 22 a 40 ms no p99,9 e no máximo, a cargas intermédias. Por existirem em K0, vêm do ambiente e não das camadas medidas; não afetam o p99 abaixo de cerca de 300 mensagens/s por sensor. Na campanha, os maiores valores nas execuções de custo foram de 0,5 ms em K0. A partir de 400 mensagens/s por sensor (20 000 mensagens/s), a latência mediana sobe de cerca de 0,2 para 7–14 ms em todas as configurações, sem perda, com o *gateway* entre 40 % (K0) e 63 % (K8) de processador, longe da saturação, e volta a descer quando a carga aumenta. O salto acontece à mesma taxa por sensor em todas as

configurações; foi registado, mas a sua origem não foi determinada. Afeta a latência acima de R_{alvo} (200 mensagens/s por sensor) e fica abaixo de $L_{\max}$, pelo que não muda nenhum veredicto, mas a latência na subida de carga acima desse ponto é um majorante.

- **Serviços alheios.** O Docker esteve ativo no Raspberry Pi durante o teste de fumo e os pilotos, e foi parado para a campanha, tal como o ambiente gráfico; os pilotos só serviram para fixar parâmetros, não entram nos resultados.
- **O próprio laboratório tinha defeitos.** Três foram encontrados nas duas primeiras campanhas completas e corrigidos antes da terceira, a única usada (Secção 3.7). O mais grave fazia contar como impedidos ataques que não chegaram a ser feitos; foi detetado por o ataque de inundação aparecer impedido em K7, que não tem limitação. Na campanha usada, verificou-se em todas as células dos ataques internos que o atacante se ligou como o dispositivo comprometido e enviou mensagens.

9.3 Validade externa

- Um só modelo de *hardware* restrito (Raspberry Pi 5), um só *broker* (Mosquitto) e uma rede local. Um microcontrolador sem sistema operativo, ou uma rede com perdas, podem ordenar as configurações de outra forma.
- **Sem AES em *hardware*** foi emulado desligando extensões no OpenSSL do mesmo processador, o que não reproduz a memória nem a *cache* de um processador mais simples.
- Carga sintética: no nível 2, mensagens de 16 octetos de conteúdo a taxa constante, QoS 0, 50 sensores; o tamanho só foi variado no nível 1. Com mensagens maiores, o peso do envelope e a vantagem do AES em *hardware* crescem (Secção 4.4).

9.4 Limitações do protótipo

As limitações da Parte 1, Secção 6.11 mantêm-se, com uma alteração: o reinício do recetor deixou de reabrir a janela anti-repetição (Secção 2), à custa de exigir uma rotação de chaves depois de cada reinício. Continuam por resolver a distribuição de chaves fora do laboratório, o armazenamento de chaves (só em memória), a supressão e o atraso de mensagens por um *broker* ativo, e a confidencialidade e a prova de origem em simultâneo (cifra seguida de assinatura).

10 Conclusões

As medições no Raspberry Pi 5 respondem às cinco perguntas da Parte 1:

QP1. Custo de cada camada. À carga nominal, o transporte (mTLS, autenticação e ACL) custa 7,9 μs de processador por mensagem e o envelope AES-GCM 7,2 μs; a validade, a anti-repetição, a restrição chave–tópico e a limitação de tráfego custam, juntas, menos de 3 μs. A configuração recomendada para a telemetria (K8) gasta 41 μs por mensagem, contra 24 μs sem proteção, com p99 de 262 μs. As assinaturas Ed25519 custam mais 151 μs por mensagem.

QP2. O que cada camada impede. O transporte e o envelope impedem, cada um por si, quase todos os ataques de um estranho ($S = 0{,}64$ cada), mas ataques diferentes: só o envelope protege perante o *broker*, só o transporte impede a usurpação de identidades. Os ataques de um dispositivo comprometido no seu próprio tópico só são impedidos pela combinação da política de modo, da validade temporal (a janela sozinha não chega) e da limitação de tráfego. K8, K9 e K11 impedem os treze ataques da matriz.

QP3. Tamanho e saturação. Até alguns KiB, o custo do envelope é dominado pela parte fixa (5,6 μs a 16 B, 13 μs a 4 KiB). O débito máximo é limitado pelo processador do *gateway*: 60 000 mensagens/s sem proteção, 30 000 com K8 e 5 000 com assinaturas, o que deixa K9 abaixo dos 10 000 mensagens/s da telemetria.

QP4. Aceleração por *hardware*. Com as extensões ARMv8, o AES-GCM é o mais barato em todos os tamanhos; sem elas, fica sete vezes mais caro a 4 KiB e o ChaCha20-Poly1305, que só usa NEON, passa a custar quatro vezes menos.

QP5. Configuração recomendada. K8 para a telemetria, sensível ou não (K11 num processador sem AES em *hardware*), e K9 para as ordens a atuadores, a única com prova de origem. A recomendação da Parte 1 confirma-se e não muda com outros pesos de custo ou de ataque.

Das cinco hipóteses, três confirmam-se (H1, com uma ressalva quanto ao débito; H2; H3). H4 é refutada: em ligações MQTT duradouras, o custo do TLS está em cada mensagem, tanto como o do envelope, e não na negociação. H5 não se confirma: com AES em *hardware*, a limitação antes da criptografia não poupa processador mensurável ao *gateway*.

Contributo para a lacuna L5. A Parte 1 identificou a falta de medições do custo e do benefício de cada camada, isoladamente e em combinação, em *hardware* restrito. Este trabalho mede-as num Raspberry Pi 5, com doze configurações que só diferem numa camada de cada vez, 120 execuções de custo, uma subida de carga por configuração, 468 ataques e 39 cenários com controlo negativo, todos guardados com o ambiente e reprodutíveis por um só comando. Os resultados mostram que as camadas baratas (validade, anti-repetição, ligação ao tópico, limitação) são as que fecham os ataques de um dispositivo comprometido, e que o custo de as ter todas sobre o transporte e o envelope é de cerca de 2 μs por mensagem.

Trabalho futuro. Cifra seguida de assinatura, para dar confidencialidade e prova de origem ao mesmo tempo; distribuição e armazenamento de chaves fora do laboratório; medição do custo no emissor num dispositivo restrito real (microcontrolador); um *broker* ativo que termine TLS; uma configuração com assinaturas e sem limitação, para medir H5 onde a verificação é cara; a origem do salto de latência registado a 400 mensagens/s por sensor; e a energia.

Referências

A Reprodução dos resultados

O repositório contém o código, os dados brutos de cada campanha (`results/`) e os programas de análise. Os passos para repetir as medições num Raspberry Pi estão em `docs/raspberry-pi.md`; o procedimento e o significado de cada ficheiro, em `docs/evaluation.md`.

- **Refazer as tabelas e figuras a partir dos dados brutos:** `tools/analysis/analyze.py results/<campanha>` e, nesta pasta, `make dados PI=<campanha>` seguido de `make`.
- **Repetir as medições:** `loadgen.py campaign` com a mesma semente e o mesmo `params.toml` (guardados em cada campanha).

A Tabela 13 identifica a campanha de que vêm os resultados.

Tabela 13. Campanha usada neste relatório.

Nome	`pi5-20261113` (`results/pi5-20261113/`)
Plataforma	Raspberry Pi 5, 8 GB (*broker* e *gateway*); computador de secretária (dois agentes de sensores e um atacante)
Data	12–13 de novembro de 2026, 23:37 a 04:32 (UTC)
Commit	`79c4878`, árvore limpa
Semente	20261201
Execuções	120 de custo (12 configurações × 10); subida de carga com 3 repetições por configuração (427 patamares); matriz de ataques 3 × 156; 39 cenários; *microbenchmarks* com 10 repetições em três variantes de aceleração
Testes no mesmo *commit*	`results/validation/pi5-20261113/`

Plataforma de referência

Em falta: `gerado/desktop/summary.tex`
(`make dados` depois da campanha)

Em falta: `gerado/desktop/criterion.tex`
(`make dados` depois da campanha)